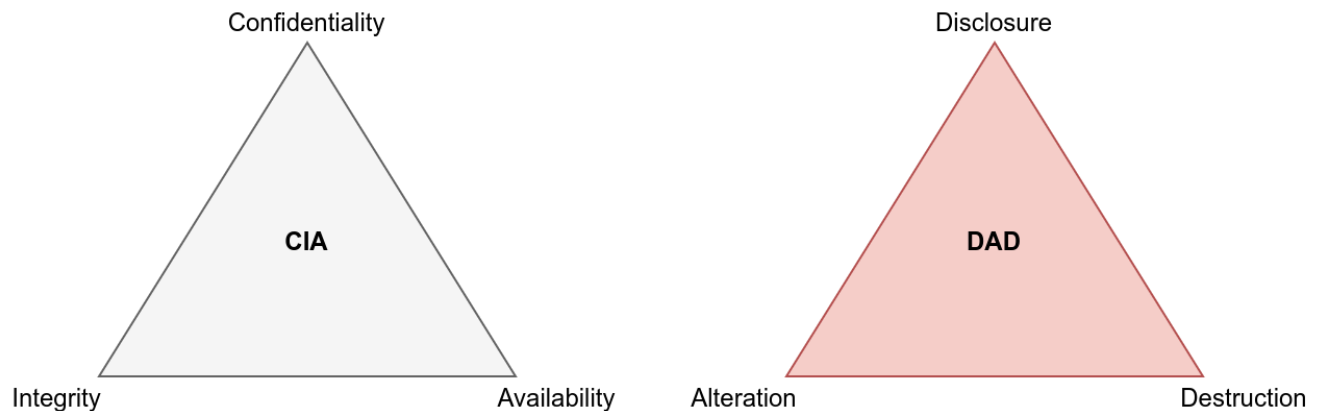


Protocols and Servers 2



Sniffing Attack

Sniffing attack refers to using a network packet capture tool to collect information about the target

A sniffing attack can be conducted using an Ethernet (802.3) network card, provided that the user has proper permissions (root permissions on Linux and administrator privileges on MS Windows). There are many programs available to capture network packets. We consider the following:

1. **Tcpdump** is a free open source command-line interface (CLI) program that has been ported to work on many operating systems.
2. **Wireshark** is a free open source graphical user interface (GUI) program available for several operating systems, including Linux, macOS and MS Windows.
3. **Tshark** is a CLI alternative to Wireshark.

Consider a user checking his email messages using POP3. First, we are going to use Tcpdump to attempt to capture the username and password. In the terminal output below, we used the command `sudo tcpdump port 110 -A`

```

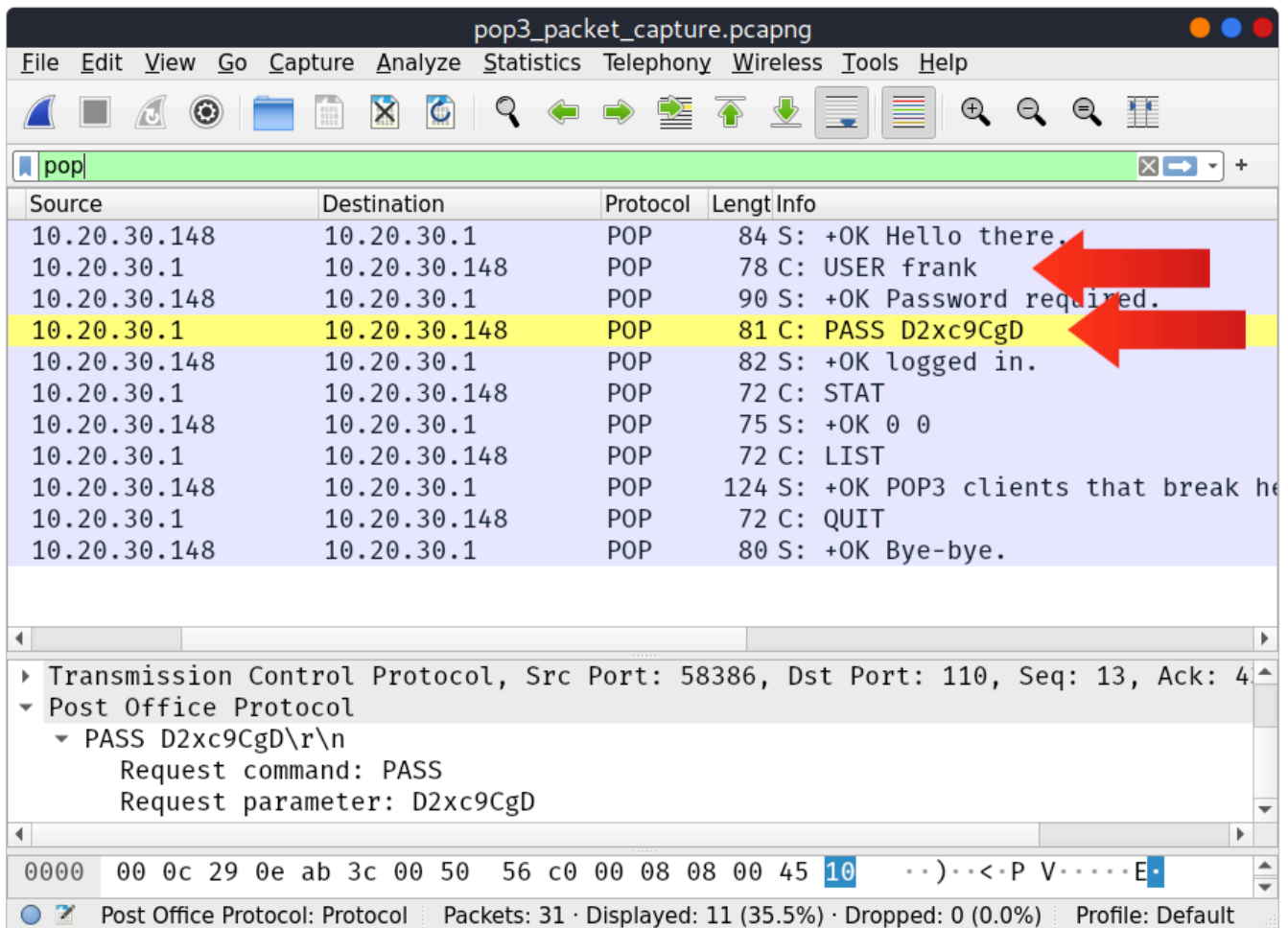
pentester@TryHackMe$ sudo tcpdump port 110 -A
[...]
09:05:15.132861 IP 10.20.30.1.58386 > 10.20.30.148.pop3: Flags [P.], seq 1:13, ack
E..@.V@.@.g.
...
.....n.....".....
.;....}.USER frank

09:05:15.133465 IP 10.20.30.148.pop3 > 10.20.30.1.58386: Flags [.], ack 13, win 51
E..4..@.@.O~
...
....n.....".....?P.....
...i.;..
09:05:15.133610 IP 10.20.30.148.pop3 > 10.20.30.1.58386: Flags [P.], seq 19:43, ack
E..L..@.@.Oe
...
....n.....".....<-.....
...i.;..+OK Password required.

09:05:15.133660 IP 10.20.30.1.58386 > 10.20.30.148.pop3: Flags [.], ack 43, win 50
E..4.W@.@.g.
...
.....n.....".....??.....
.;....i
09:05:22.852695 IP 10.20.30.1.58386 > 10.20.30.148.pop3: Flags [P.], seq 13:28, ack
E..C.X@.@.g.
...
.....n.....".....6.....
.<.....iPASS D2xc9CgD
[...]

```

use Wireshark to achieve the same results. In the Wireshark window below, we can see that we have entered `pop` in the filter field. Now that we've filtered just the traffic we're interested in, we can see a username and password were captured.



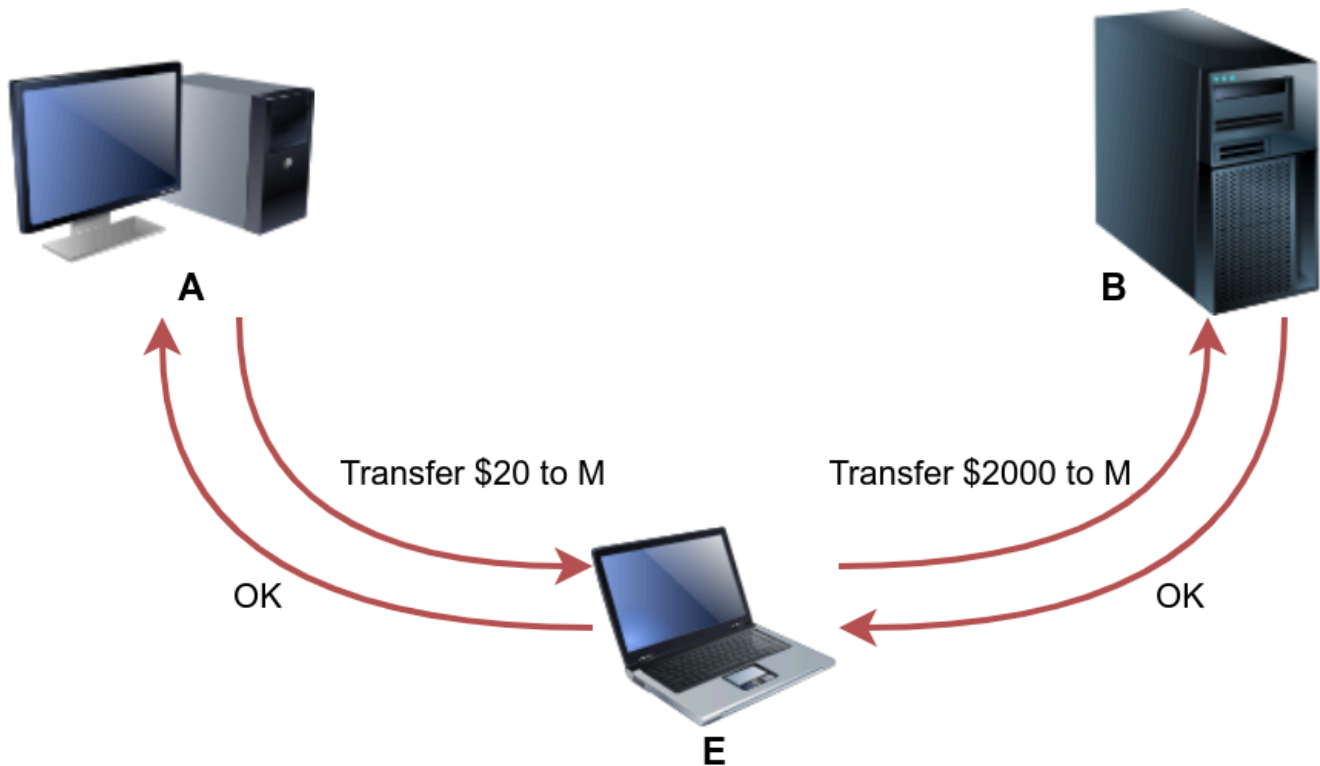
any protocol that uses cleartext communication is susceptible to this kind of attack. The only requirement for this attack to succeed is to have access to a system between the two communicating systems.

mitigation lies in adding an encryption layer on top of any network protocol

MITM

A Man-in-the-Middle (MITM) attack occurs when a victim (A) believes they are communicating with a legitimate destination (B) but is unknowingly communicating with an attacker (E). In the figure below, we have A requesting the transfer of \$20 to M; however, E altered this message and replaced the original value with a new one. B received the modified message and acted on it.

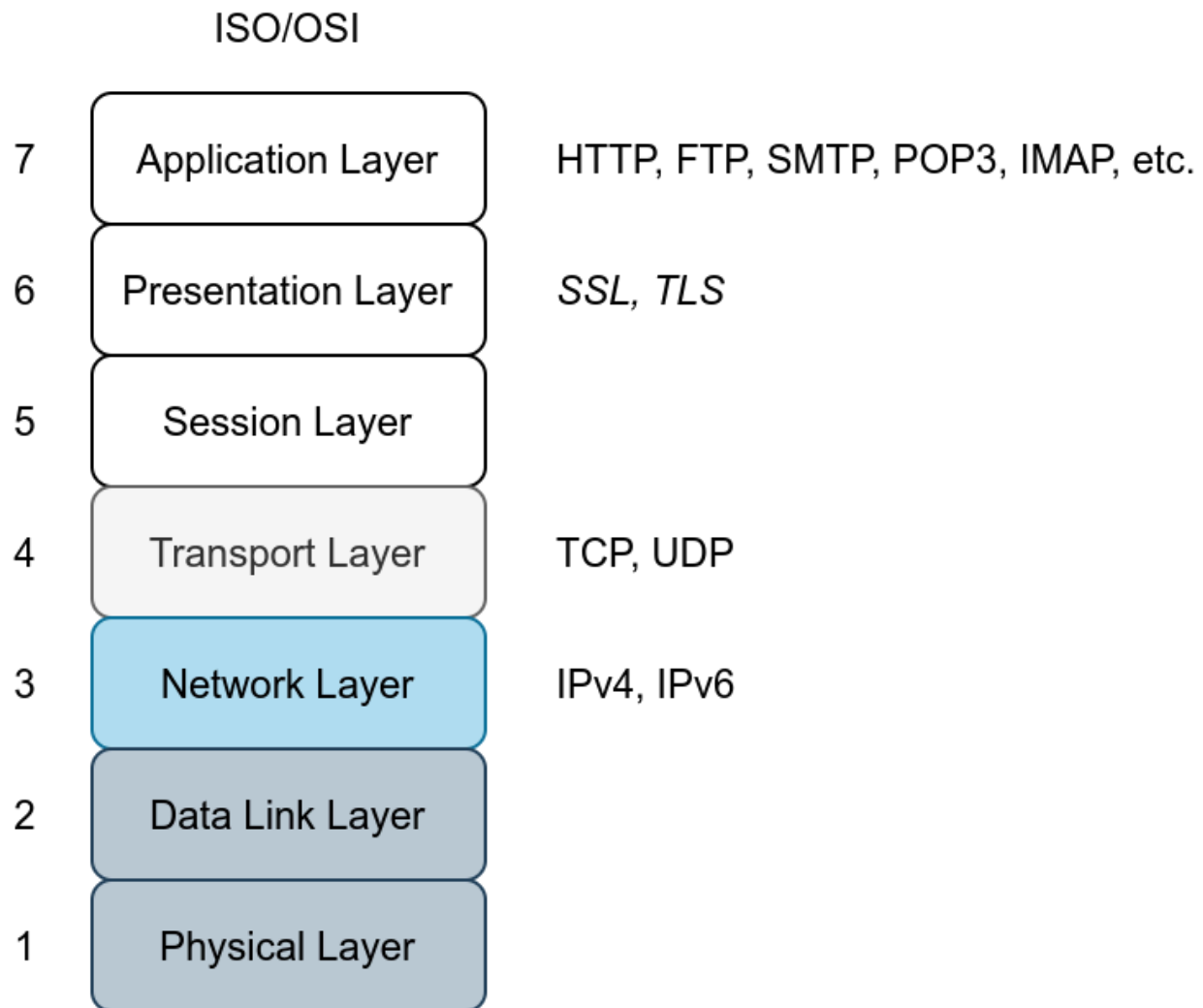
Man-in-the-Middle (MITM)



Many tools would aid you in carrying out such an attack, such as [Ettercap\(opens in new tab\)](#) and [Bettercap\(opens in new tab\)](#).

Mitigation against this attack requires the use of cryptography. The solution lies in proper authentication along with encryption or signing of the exchanged messages

TLS



An existing cleartext protocol can be upgraded to use encryption via SSL/TLS. We can use TLS to upgrade HTTP, FTP, SMTP, POP3, and IMAP, to name a few.

Protocol	Default Port	Secured Protocol	Default Port with TLS
HTTP	80	HTTPS	443
FTP	21	FTPS	990
SMTP	25	SMTPS	465
POP3	110	POP3S	995
IMAP	143	IMAPS	993

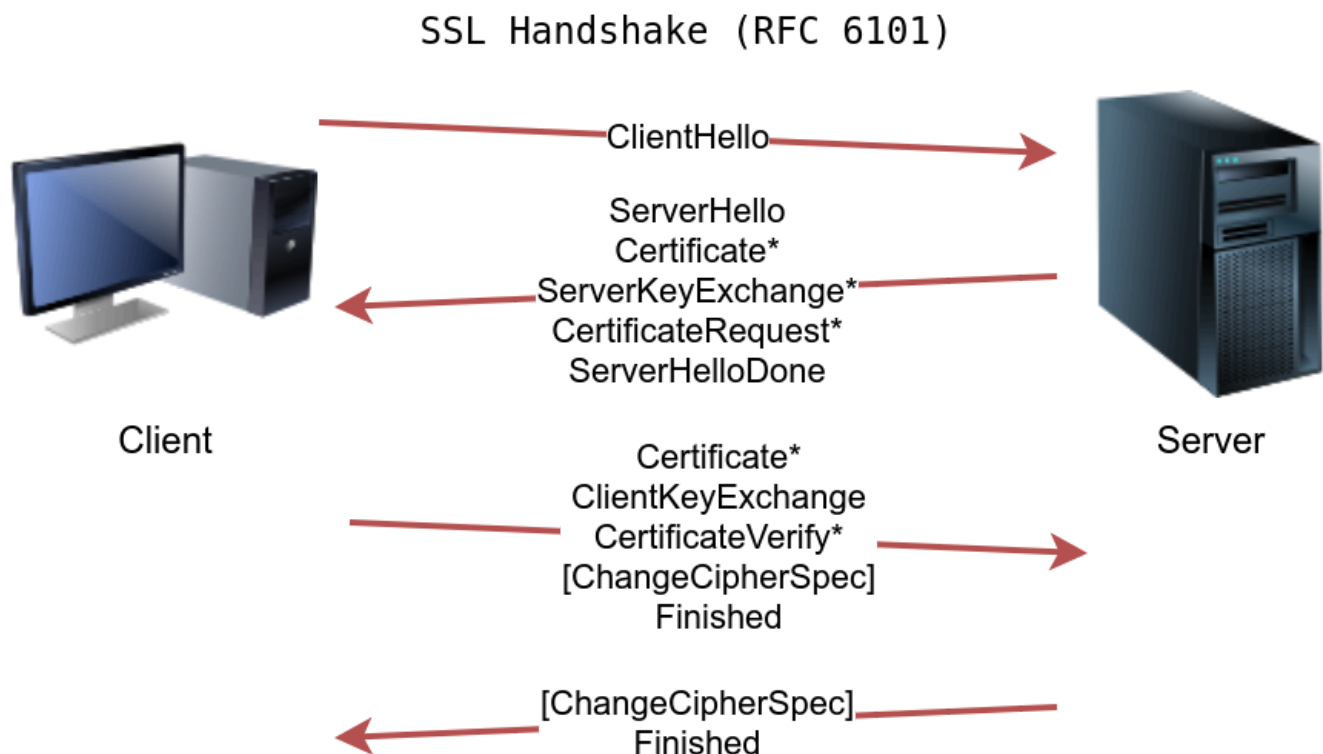
Considering the case of HTTP. Initially, to retrieve a web page over HTTP, the web browser would need at least perform the following two steps:

1. Establish a TCP connection with the remote web server
2. Send HTTP requests to the web server, such as GET and POST requests.

HTTPS requires an additional step to encrypt the traffic. The new step takes place after establishing a TCP connection and before sending HTTP requests. This extra step can be inferred from the ISO/OSI model in the image presented earlier. Consequently, HTTPS requires at least the following three steps:

1. Establish a TCP connection
2. **Establish SSL/TLS connection**
3. Send HTTP requests to the webserver

To establish an SSL/TLS connection, the client needs to perform the proper handshake with the server. Based on [RFC 6101](#)(opens in new tab), the SSL connection establishment will look like the figure below.



1. The client sends a ClientHello to the server to indicate its capabilities, such as supported algorithms.
2. The server responds with a ServerHello, indicating the selected connection parameters. The server provides its certificate if server authentication is required. The certificate is a digital file to identify itself; it is usually digitally signed by a third party. Moreover, it might

send additional information necessary to generate the master key, in its ServerKeyExchange message, before sending the ServerHelloDone message to indicate that it is done with the negotiation.

3. The client responds with a ClientKeyExchange, which contains additional information required to generate the master key. Furthermore, it switches to use encryption and informs the server using the ChangeCipherSpec message.
4. The server switches to use encryption as well and informs the client in the ChangeCipherSpec message.

Secure Shell

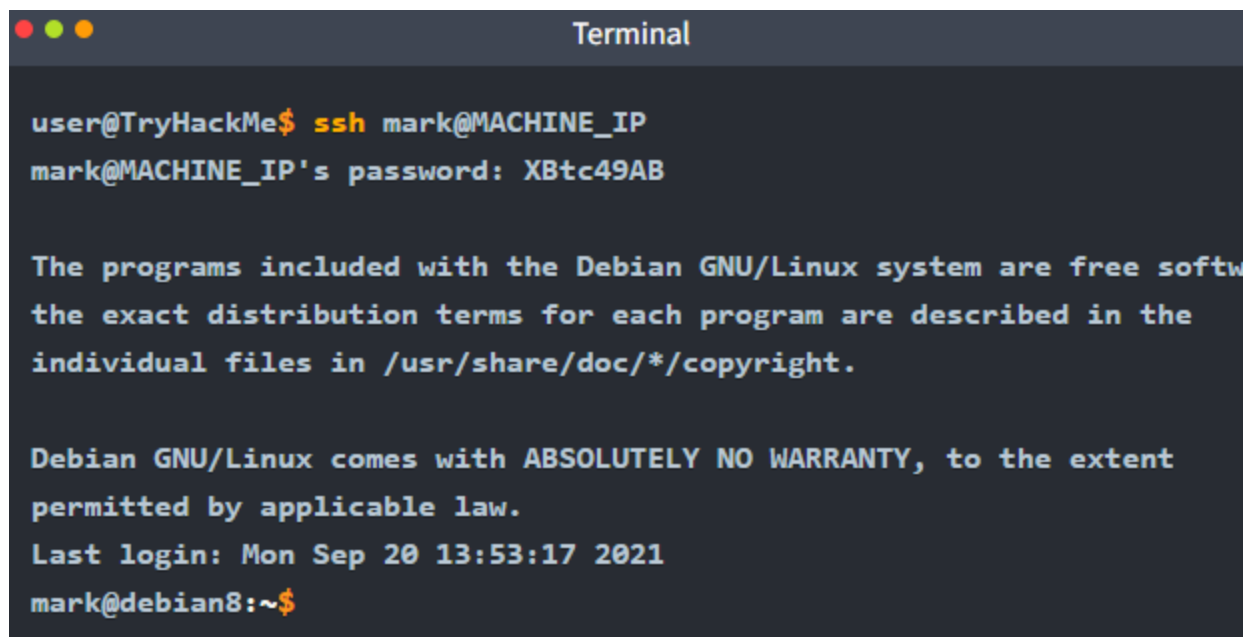
lets you securely connect to another system over the network and execute commands on the remote system.

1. You can confirm the identity of the remote server
2. Exchanged messages are encrypted and can only be decrypted by the intended recipient
3. Both sides can detect any modification in the messages

To use SSH, you need an SSH server and an SSH client. The SSH server listens on port 22 by default. The SSH client can authenticate using:

- A username and a password
- A private and public key (after the SSH server is configured to recognise the corresponding public key)

On Linux, macOS, and MS Windows builds after 2018, you can connect to an SSH server using the following command `ssh username@MACHINE_IP`. This command will try to connect to the server of IP address `MACHINE_IP` with the login name `username`



```
Terminal

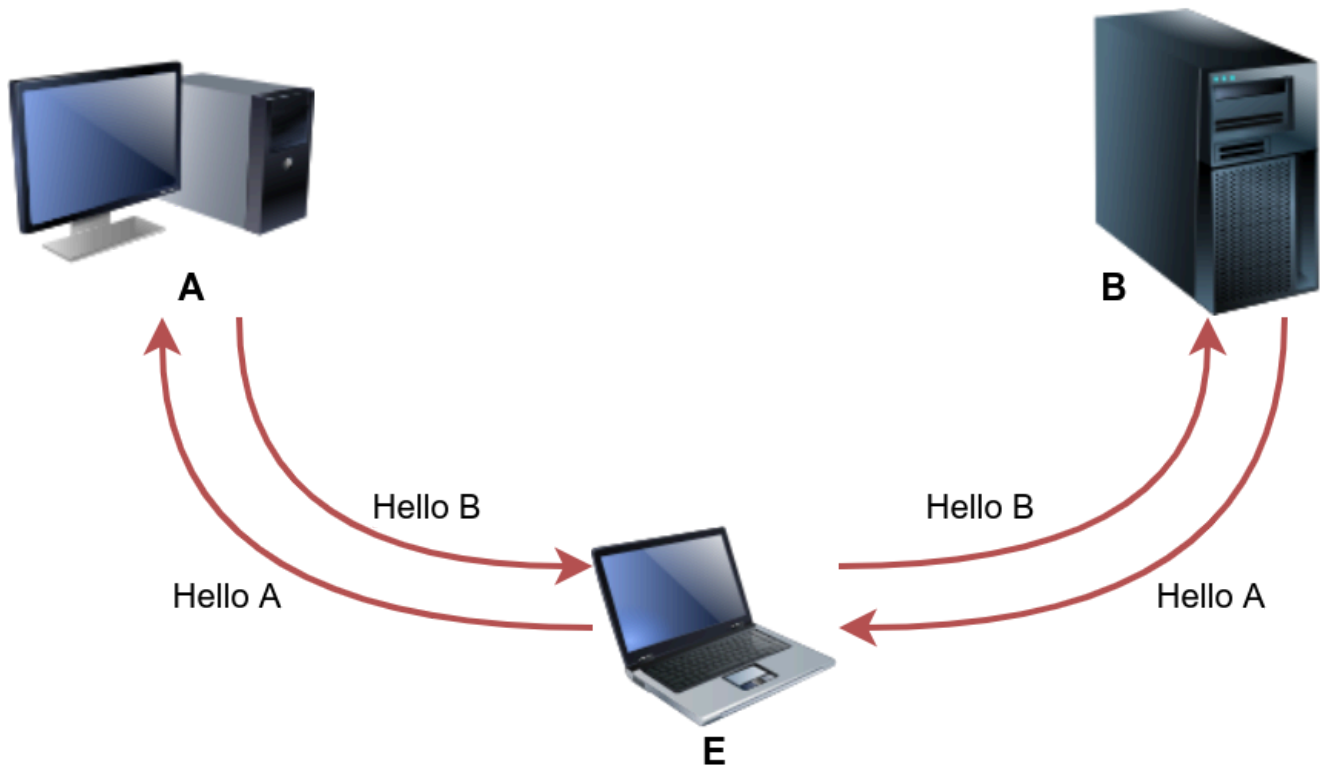
user@TryHackMe$ ssh mark@MACHINE_IP
mark@MACHINE_IP's password: XBtc49AB

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.

Last login: Mon Sep 20 13:53:17 2021
mark@debian8:~$
```

Man-in-the-Middle (MITM)



We can use SSH to transfer files using SCP (Secure Copy Protocol) based on the SSH protocol.

An example of the syntax is as follows: `scp mark@MACHINE_IP:/home/mark/archive.tar.gz ~`.

This command will copy a file named `archive.tar.gz` from the remote system located in the `/home/mark` directory to `~`, i.e., the root of the home directory of the currently logged-in user.

Another example syntax is `scp backup.tar.bz2 mark@MACHINE_IP:/home/mark/`

This command will copy the file `backup.tar.bz2` from the local system to the directory `/home/mark/` on the remote system.

```
Terminal

user@TryHackMe$ scp document.txt mark@MACHINE_IP:/home/mark
mark@MACHINE_IP's password:
document.txt                                     100% 1997KB  70.4M
```

Practical:


```

root@ip-10-128-124-29:~# ssh mark@10.128.173.13
The authenticity of host '10.128.173.13 (10.128.173.13)' can't be established.
ECDSA key fingerprint is SHA256:9YI/fsZisEv3581v7B3pkj+3MeFiGydmPV+5pvHf61M.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.128.173.13' (ECDSA) to the list of known hosts.
mark@10.128.173.13's password:
Welcome to Ubuntu 20.04.6 LTS (GNU/Linux 5.15.0-119-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Tue 05 May 2026 05:50:21 PM UTC

System load:  0.0               Processes:            120
Usage of /:   66.2% of 6.50GB   Users logged in:     0
Memory usage: 32%              IPv4 address for eth0: 10.128.173.13
Swap usage:   0%

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment.

   https://ubuntu.com/engage/secure-kubernetes-at-the-edge

Expanded Security Maintenance for Applications is not enabled.

13 updates can be applied immediately.
10 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update
Your Hardware Enablement Stack (HWE) is supported until April 2025.

Last login: Mon Sep 20 13:36:07 2021 from 10.20.30.1
Last login: Mon Sep 20 13:36:07 2021 from 10.20.30.1
mark@ip-10-128-173-13:~$ uname -r
5.15.0-119-generic

```

uname -r is a command to find the kernel release

scp used to securely download a file

```

mark@ip-10-128-173-13:~$ scp book.txt mark@10.128.173.13:/home/mark
The authenticity of host '10.128.173.13 (10.128.173.13)' can't be established.
ECDSA key fingerprint is SHA256:9YI/fsZisEv3581v7B3pkj+3MeFiGydmPV+5pvHf61M.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.128.173.13' (ECDSA) to the list of known hosts.
mark@10.128.173.13's password:
book.txt                                100% 415KB 76.7MB/s 00:00
mark@ip-10-128-173-13:~$

```

Password Attack

```
Pentester Terminal

pentester@TryHackMe$ telnet 10.128.173.13 110
Trying 10.128.173.13...
Connected to 10.128.173.13.
Escape character is '^]'.
+OK 10.128.173.13 Mail Server POP3 Wed, 15 Sep 2021 11:05:34 +0300
USER frank
+OK frank
PASS D2xc9CgD
+OK 1 messages (179) octets
STAT
+OK 1 179
LIST
+OK 1 messages (179) octets
1 179
.
RETR 1
+OK
From: Mail Server
To: Frank
subject: Sending email with Telnet
Hello Frank,
I am just writing to say hi!
.
QUIT
+OK 10.128.173.13 closing connection
Connection closed by foreign host.
```

Authentication, or proving your identity, can be achieved through one of the following, or a combination of two:

1. Something you *know*, such as password and PIN code.
2. Something you *have*, such as a SIM card, RFID card, and USB dongle.
3. Something you *are*, such as fingerprint and iris.

Attacks against passwords are usually carried out by:

1. Password Guessing: Guessing a password requires some knowledge of the target, such as their pet's name and birth year.
2. Dictionary Attack: This approach expands on password guessing and attempts to include all valid words in a dictionary or a wordlist.

3. Brute Force Attack: This attack is the most exhaustive and time-consuming where an attacker can go as far as trying all possible character combinations, which grows fast (exponential growth with the number of characters).

The general command-line syntax is: `hydra -l username -P wordlist.txt server service` where we specify the following options:

- `-l username` : `-l` should precede the `username` , i.e. the login name of the target.
- `-P wordlist.txt` : `-P` precedes the `wordlist.txt` file, which is a text file containing the list of passwords you want to try with the provided username.
- `server` is the hostname or IP address of the target server.
- `service` indicates the service which you are trying to launch the dictionary attack.

Consider the following concrete examples:

- `hydra -l mark -P /usr/share/wordlists/rockyou.txt 10.128.173.13 ftp` will use `mark` as the username as it iterates over the provided passwords against the FTP server.
- `hydra -l mark -P /usr/share/wordlists/rockyou.txt ftp://10.128.173.13` is identical to the previous example. `10.128.173.13 ftp` is the same as `ftp://10.128.173.13` .
- `hydra -l frank -P /usr/share/wordlists/rockyou.txt 10.128.173.13 ssh` will use `frank` as the user name as it tries to login via SSH using the different passwords.

There are some extra optional arguments that you can add:

- `-s PORT` to specify a non-default port for the service in question.
- `-V` or `-vV` , for verbose, makes Hydra show the username and password combinations that are being tried. This verbosity is very convenient to see the progress, especially if you are still not confident of your command-line syntax.
- `-t n` where `n` is the number of parallel connections to the target. `-t 16` will create 16 threads used to connect to the target.
- `-d` , for debugging, to get more detailed information about what's going on. The debugging output can save you much frustration; for instance, if Hydra tries to connect to a closed port and timing out, `-d` will reveal this right away.

Mitigation against such attacks can be sophisticated and depends on the target system. A few of the approaches include:

- Password Policy: Enforces minimum complexity constraints on the passwords set by the user.
- Account Lockout: Locks the account after a certain number of failed attempts.

- Throttling Authentication Attempts: Delays the response to a login attempt. A couple of seconds of delay is tolerable for someone who knows the password, but they can severely hinder automated tools.
- Using CAPTCHA: Requires solving a question difficult for machines. It works well if the login page is via a graphical user interface (GUI). (Note that CAPTCHA stands for Completely Automated Public Turing test to tell Computers and Humans Apart.)
- Requiring the use of a public certificate for authentication. This approach works well with SSH, for instance.
- Two-Factor Authentication: Ask the user to provide a code available via other means, such as email, smartphone app or SMS.
- There are many other approaches that are more sophisticated or might require some established knowledge about the user, such as IP-based geolocation.

Practical Example:

to get the password for the IMAP service running on the IP 10.128.173.13

using command: `hydra -l lazle -P /usr/share/wordlists/rockyou.txt 10.128.173.13 imap`

```
^Croot@ip-10-128-124-29:~# hydra -l lazle -P /usr/share/wordlists/rockyou.txt 10.128.173.13 imap
Hydra v9.0 (c) 2019 by van Hauser/THC - Please do not use in military or secret service organizations, or for illegal
purposes.

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-05-05 19:29:08
[INFO] several providers have implemented cracking protection, check with a small wordlist first - and stay legal!
[WARNING] Restorefile (you have 10 seconds to abort... (use option -I to skip waiting)) from a previous session found,
to prevent overwriting, ./hydra.restore
[DATA] max 16 tasks per 1 server, overall 16 tasks, 14344398 login tries (l:1/p:14344398), ~896525 tries per task
[DATA] attacking imap://10.128.173.13:143/
[143][imap] host: 10.128.173.13 login: lazle password: butterfly
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-05-05 19:29:40
```

Summary

It is good to remember the default port number for common protocols

Protocol	TCP Port	Application(s)	Data Security
FTP	21	File Transfer	Cleartext
FTPS	990	File Transfer	Encrypted
HTTP	80	Worldwide Web	Cleartext
HTTPS	443	Worldwide Web	Encrypted
IMAP	143	Email (MDA)	Cleartext
IMAPS	993	Email (MDA)	Encrypted
POP3	110	Email (MDA)	Cleartext
POP3S	995	Email (MDA)	Encrypted
SFTP	22	File Transfer	Encrypted
SSH	22	Remote Access and File Transfer	Encrypted
SMTP	25	Email (MTA)	Cleartext
SMTPS	465	Email (MTA)	Encrypted
Telnet	23	Remote Access	Cleartext

Hydra remains a very efficient tool that you can launch from the terminal to try the different passwords.

Option	Explanation
<code>-l username</code>	Provide the login name
<code>-P WordList.txt</code>	Specify the password list to use
<code>server service</code>	Set the server address and service to attack
<code>-s PORT</code>	Use in case of non-default service port number
<code>-V</code> or <code>-vV</code>	Show the username and password combinations being tried
<code>-d</code>	Display debugging output if the verbose output is not helping